Software Requirements Specification

countdown: An X Window Countdown Timer

Countdown Software Project

Prepared by: J. R. Evans

Document identifier:	CD-SRS-001
Revision:	1.0
Date:	June 13, 2026
Status:	Released
Governing standard:	IEEE Std 830-1998 (SRS)

This document is prepared in accordance with the principles of rational documentation articulated in D. L. Parnas and P. C. Clements, "A Rational Design Process: How and Why to Fake It," IEEE Transactions on Software Engineering, vol. SE-12, no. 2, pp. 251–257, February 1986.

Contents

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) states the externally observable requirements on the `countdown` program. It is the contract against which the Software Design Document (CD-SDD-001) is written and against which the Software Test Plan (CD-STP-001) and its automated harness verify the delivered program.

1.2 Scope

`countdown` is a single-window X Window System application that displays the time remaining until a fixed future instant and updates that display live. The SRS covers the program tangled from `countdown.w`. It does not cover the X server, the operating system's time-zone database, or the CWEB tools themselves, except to state the program's reliance on them.

1.3 Definitions and acronyms

Term	Meaning
Target instant	The fixed future date and time to which the program counts down.
Armed	Started but not yet running; shows the full remaining duration.
Running	Counting down live.
Paused	Display frozen at the value captured when the user paused.
Arrived	The remaining time has reached zero.
CWEB	Knuth's literate-programming system; `ctangle` extracts C, `cweave` extracts TEX.
Xlib	The C-language client interface to the X Window System.
POSIX	IEEE Std 1003.1 operating-system interface.

1.4 Rational-process note

In the spirit of Parnas and Clements, the requirements below are presented as a clean, numbered hierarchy as though derived top-down. They were in fact refined alongside the design; the idealised presentation is retained because it is the form most useful to a reader.

1.5 Requirement identifier scheme

Each requirement carries an identifier `REQ-`*`C`*`-`*`nn`* where C is a category letter—**F**unctional, **G**UI, **T**iming, **D**esign-constraint, or **B**uild—and nn is a serial number. Identifiers are stable and are referenced verbatim by CD-STP-001 and by the automated test harness.

2 Overall description

2.1 Product perspective

`countdown` is a self-contained, standalone program. It depends on an X server for display and input, and on the POSIX time facilities of the host for the current time and time-zone conversion. It has no persistent storage, no configuration file, and no network interface.

2.2 Product functions (summary)

The program converts a target calendar string to an absolute instant, repeatedly computes the seconds remaining until that instant, formats them as days and a clock, and paints them in an X window, reacting to keyboard and mouse input to start, pause, resume, and quit.

2.3 User characteristics

The user is anyone able to launch a program in an X session. No specialised knowledge is assumed beyond reading a window and pressing a key or mouse button.

2.4 Constraints

The program shall be written as a literate program in CWEB and built with the project's `cweb.sh` helper. It shall link only against core Xlib. It shall run on Linux/X11 and macOS/XQuartz hosts.

2.5 Assumptions and dependencies

It is assumed that an X server is reachable via the `DISPLAY` environment variable, that the host clock is set correctly, and that the time-zone database reflects the rules in force for the target date.

3 Specific requirements

3.1 Functional requirements

ID	Requirement
`REQ-F-01`	The program shall count down to a *target instant* specified as local calendar time.
`REQ-F-02`	In the absence of a command-line argument, the target instant shall default to `2026-08-19 08:00:00` local time.
`REQ-F-03`	The program shall accept an alternate target instant as its first command-line argument, in the form `"YYYY-MM-DD HH:MM:SS"`.
`REQ-F-04`	Given a target argument that is malformed or denotes an invalid date, the program shall print a diagnostic to standard error and terminate with a non-zero exit status, without opening a window.

ID	Requirement
`REQ-F-05`	The program shall display the remaining time as a whole number of days followed by a 24-hour `HH:MM:SS` clock, with minutes and seconds zero-padded.
`REQ-F-06`	When the target instant has been reached, the remaining time shall read `0 d 00:00:00`; the program shall never display a negative remaining time.
`REQ-F-07`	Upon arrival the program shall display a status message announcing that the target instant has been reached.

3.2 Graphical user interface requirements

ID	Requirement
`REQ-G-01`	The program shall present its output in a graphical window on the X Window System using the Xlib client interface.
`REQ-G-02`	The window shall display, simultaneously, the target instant in human-readable form, the remaining time, a status line, and a one-line hint naming the available controls.
`REQ-G-03`	The user shall be able to start, pause, and resume the countdown by pressing the space bar or by clicking a mouse button in the window.
`REQ-G-04`	The user shall be able to terminate the program by pressing `q` or the Escape key.
`REQ-G-05`	The window shall be repainted in response to expose and resize (configure) events so that its contents remain correct.
`REQ-G-06`	If the preferred font is unavailable, the program shall fall back to the server's default font rather than fail.

3.3 Timing requirements

ID	Requirement
`REQ-T-01`	While running, the displayed remaining time shall advance at least once per second.
`REQ-T-02`	The remaining time shall be computed from the host's wall-clock time as obtained from the POSIX `time` service.
`REQ-T-03`	Conversion of the target calendar string to an absolute instant shall honour the host's local time zone and daylight-saving rules, as provided by the POSIX `mktime` service.
`REQ-T-04`	The program shall remain idle between updates by blocking on the POSIX `select` service with a timeout; it shall not busy-wait.

3.4 Design-constraint requirements

ID	Requirement
`REQ-D-01`	The program shall be written as a literate program in the CWEB system, with `countdown.w` as the single source of record.
`REQ-D-02`	Each woven module (CWEB section) shall contain fewer than twenty-four lines of code.
`REQ-D-03`	Every POSIX system call used by the program (directly or on the program's behalf by Xlib) shall be documented in a dedicated section of the literate program and entered in its index.
`REQ-D-04`	The compilable C source shall be a build artefact regenerable in full from `countdown.w` by `ctangle`.
`REQ-D-05`	The tangled C source shall compile cleanly under `-Wall` with no warnings.

3.5 Build and portability requirements

ID	Requirement
`REQ-B-01`	The program shall be tangled, woven, and compiled through the project `Makefile`, which drives the CWEB tools via the `cweb.sh` helper.
`REQ-B-02`	The program shall link against the core Xlib library only, with no dependency on any widget toolkit.

3.6 External interface requirements

Command-line interface. `countdown [TARGET]`, where the optional `TARGET` is a string of the form `"YYYY-MM-DD HH:MM:SS"` (REQ-F-03). Exit status is `0` on normal termination, `1` on inability to open the display, and `2` on a malformed target (REQ-F-04).

User interface. One top-level X window (REQ-G-01–REQ-G-05) with keyboard and pointer input.

Environment interface. The `DISPLAY` variable selects the X server; the `TZ` variable (or system default) selects the time zone used by `mktime`.

3.7 Non-functional requirements

Performance. Idle CPU use shall be negligible, achieved by `select`-based blocking (REQ-T-04). **Portability.** The program shall build and run on Linux/X11 and macOS/XQuartz (Section 2.4). **Readability.** The program shall be legible as a typeset document produced by `cweave` (REQ-D-01, REQ-D-02).

4 Requirements traceability

Each requirement traces upward to a ConOps capability or scenario (CD-CONOPS-001) and downward to one or more design modules (CD-SDD-001) and test cases (CD-STP-001). The condensed upward trace is:

Requirement(s)	**ConOps origin**
REQ-F-01, REQ-F-02	§6.3 capabilities 1; §7.1 Scenario 1
REQ-F-03	§7.2 Scenario 2 (custom target)
REQ-F-04	§7.4 Scenario 4 (mistyped target)
REQ-F-05–REQ-F-07	§6.3 capabilities 2 and 4; §7.3 Scenario 3
REQ-G-01–REQ-G-06	§6.1 overview; §6.4 modes of operation
REQ-T-01–REQ-T-04	§6.2 operational environment; §8.2 trade-offs
REQ-D-01–REQ-D-05	§4 justification (transparency); §8 analysis
REQ-B-01–REQ-B-02	§6.2 operational environment